

BIG DATA

DIPLOMADO DE DATOS 2026

Clase 4: Spark (Core)

Alberto Moya

albertomoya87@gmail.com

● Apache Hadoop

Search term



● apache spark

Search term



+ Add comparison

Worldwide ▼

2004 - present ▼

All categories ▼

Web Search ▼

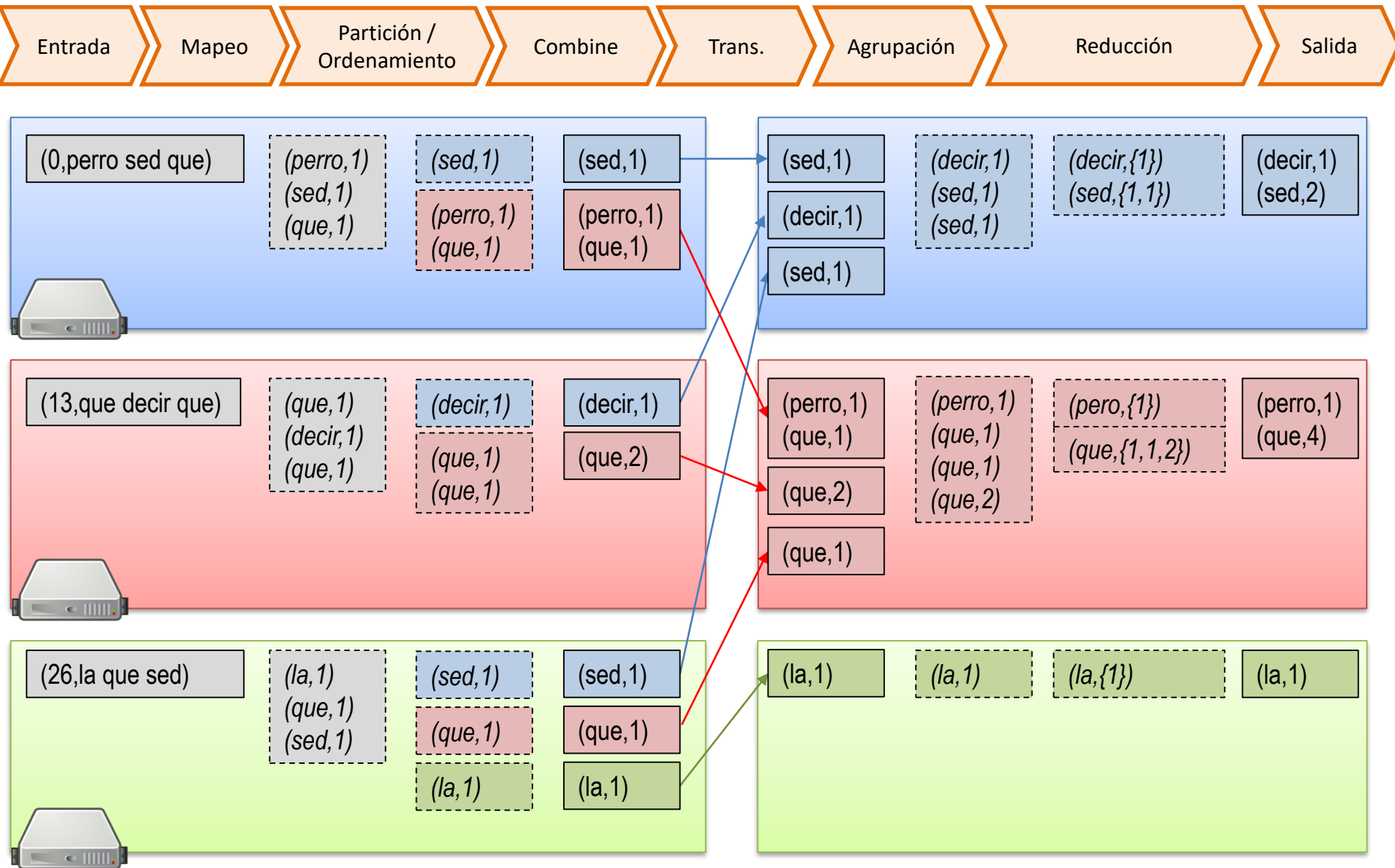
Interest over time ⓘ



¿Cuál es el principal punto débil de Hadoop en cuanto al rendimiento? ⓘ

Veremos ...

MapReduce / Hadoop





L

E L | E L

E

Entrada

Mapeo

Partición /
Ordenamiento

Combine

Trans.

Agrupación

Reducción

Salida

(0,perro sed que)

(perro,1)
(sed,1)
(que,1)

(sed,1)

(perro,1)
(que,1)

(sed,1)

(perro,1)
(que,1)

(sed,1)

(decir,1)
(sed,1)

(sed,1)

(decir,1)
(sed,1)
(sed,1)

(decir,{1})
(sed,{1,1})

(decir,1)
(sed,2)

(13,que decir que)

(que,1)
(decir,1)
(que,1)

(decir,1)

(que,1)
(que,1)

(decir,1)

(que,2)

(perro,1)
(que,1)

(que,2)

(que,1)

(perro,1)
(que,1)
(que,1)
(que,2)

(pero,{1})
(que,{1,1,2})

(perro,1)
(que,4)

(26,la que sed)

(la,1)
(que,1)
(sed,1)

(sed,1)

(que,1)

(la,1)

(sed,1)

(que,1)

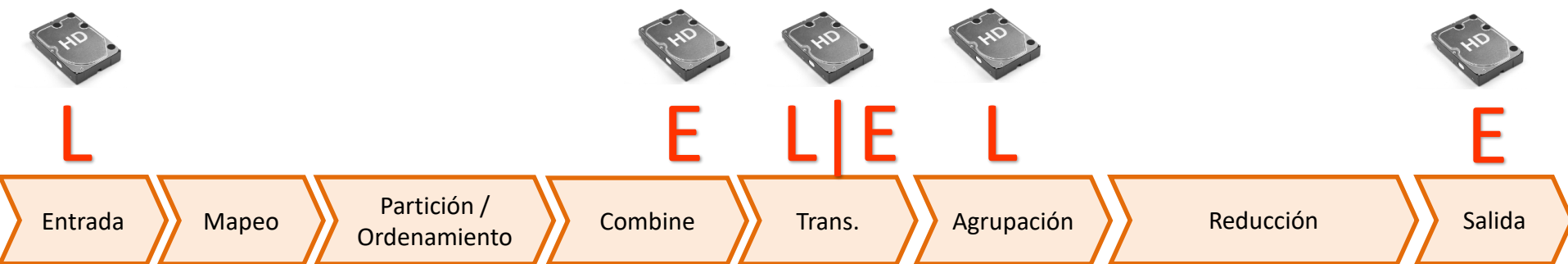
(la,1)

(la,1)

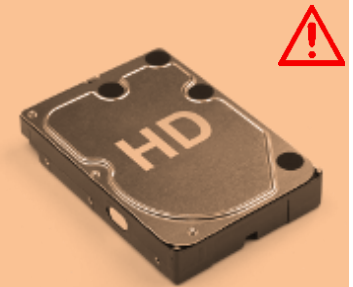
(la,1)

(la,{1})

(la,1)



MapReduce/Hadoop siempre se coordina
entre fases (Mapeo → Transmisión → Reducción)
y entre tareas (Conteo → Ordenamiento)
usando el disco duro





L



E



L|E



L



E

Entrada

Mapeo

Partición /
Ordenamiento

Combine

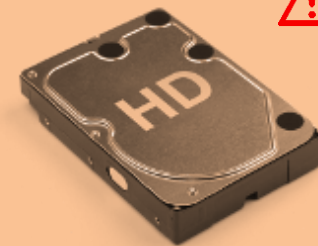
Trans.

Agrupación

Reducción

Salida

MapReduce/Hadoop siempre se coordina
entre fases (Mapeo → Transmisión → Reducción)
y entre tareas (Conteo → Ordenamiento)
usando el disco duro



En los laboratorios 1 y 2 contando palabras, vimos que toma ...

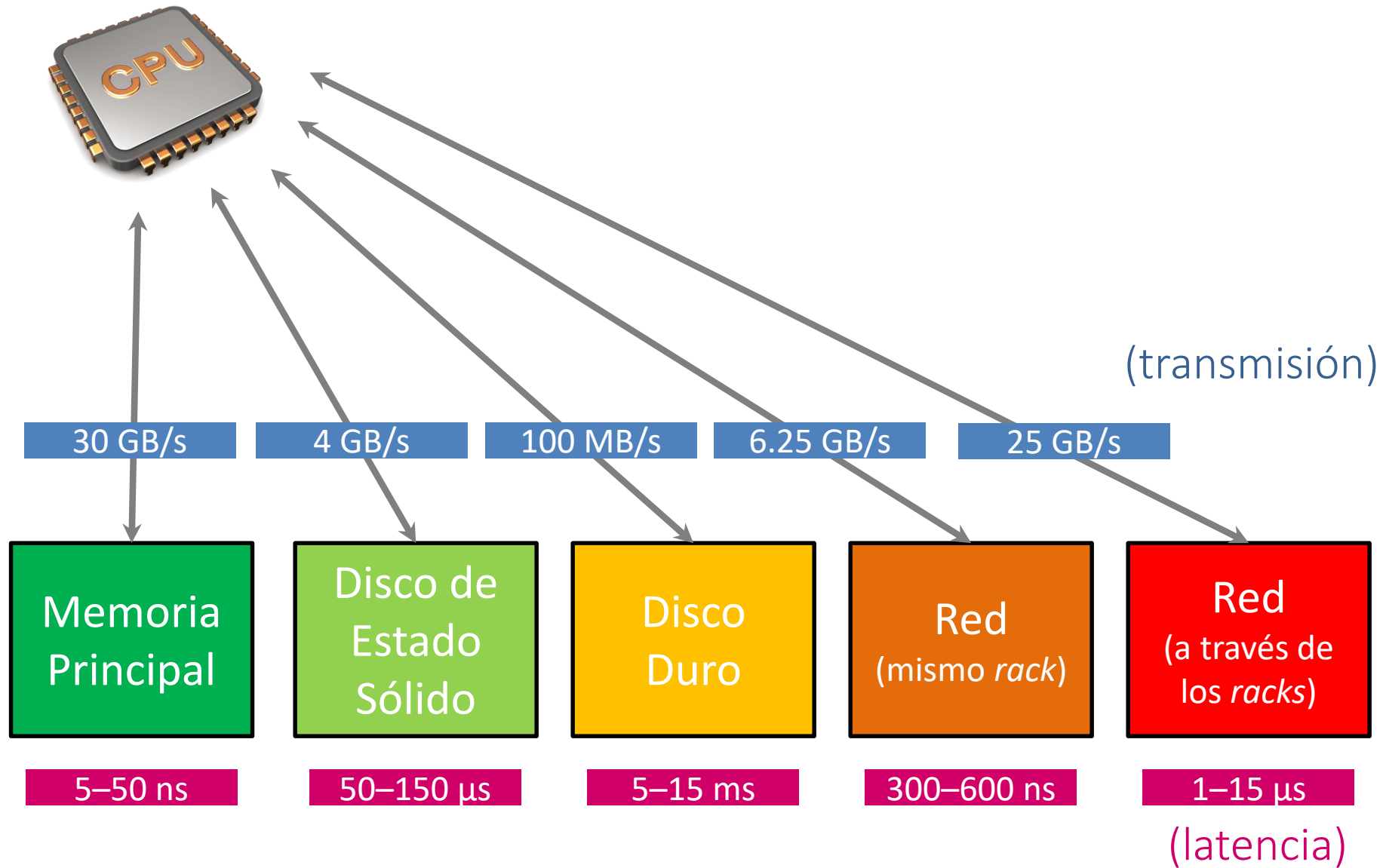
- En la memoria de una máquina: segundos
- En el disco duro de una máquina: minutos
- Usando MapReduce/Hadoop: minutos

¿Hay alguna alternativa que no hemos considerado?

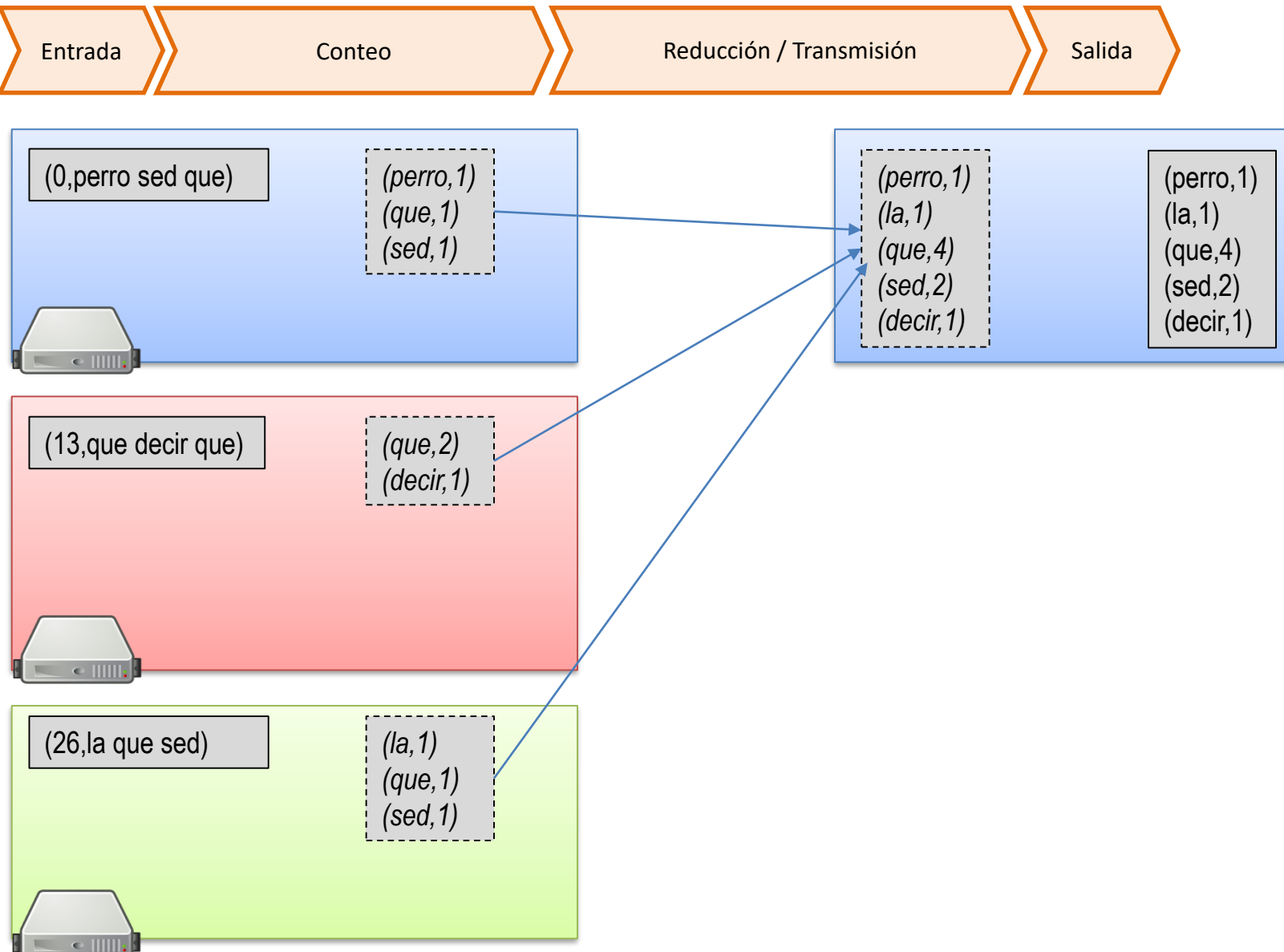


- En la memoria de varias máquinas: ???

Costos de transporte de los datos (estimaciones)



Si las palabras únicas caben en memoria ...

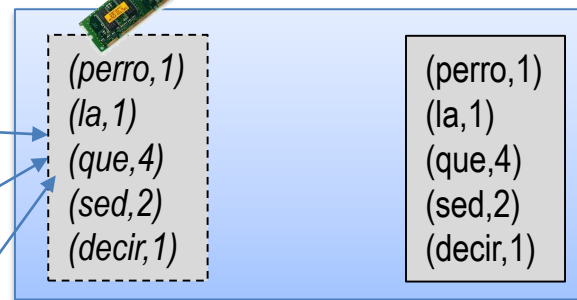
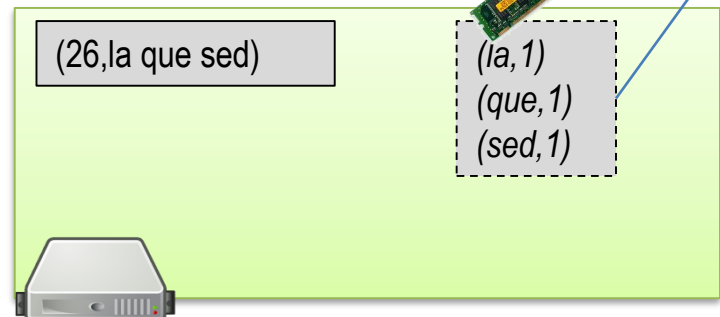
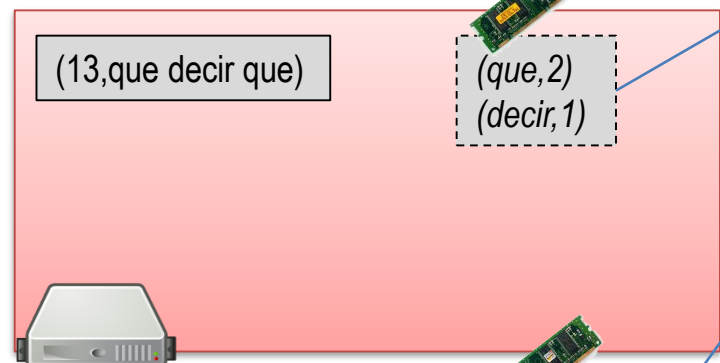
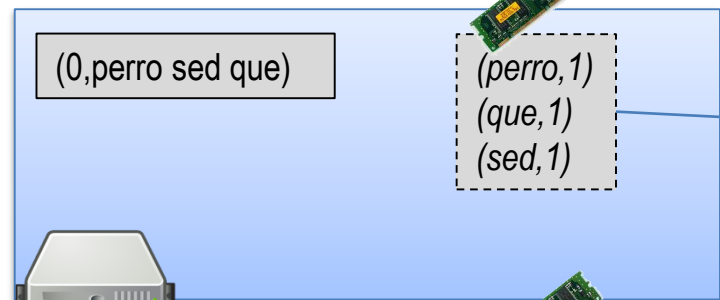




L



E



¿Y si las palabras no caben en memoria? ?

APACHE SPARK:

"RESILIENT DISTRIBUTED DATASET"

Almacenamiento: "Resilient Distributed Dataset"

Conjunto de Datos Resiliente y Distribuido



(HDFS)



RDD

Como HDFS, un RDD abstrae la distribución, la tolerancia de fallas, etc., ...
... pero un RDD también puede abstraer el disco duro / la memoria principal



L



E

Entrada

Conteo

Reducción / Transmisión

Salida

(0,perro sed que)

(perro,1)
(que,1)
(sed,1)

(13,que decir que)

(que,2)
(decir,1)

(26,la que sed)

(la,1)
(que,1)
(sed,1)

(perro,1)
(la,1)
(que,4)
(sed,2)
(decir,1)

RDD

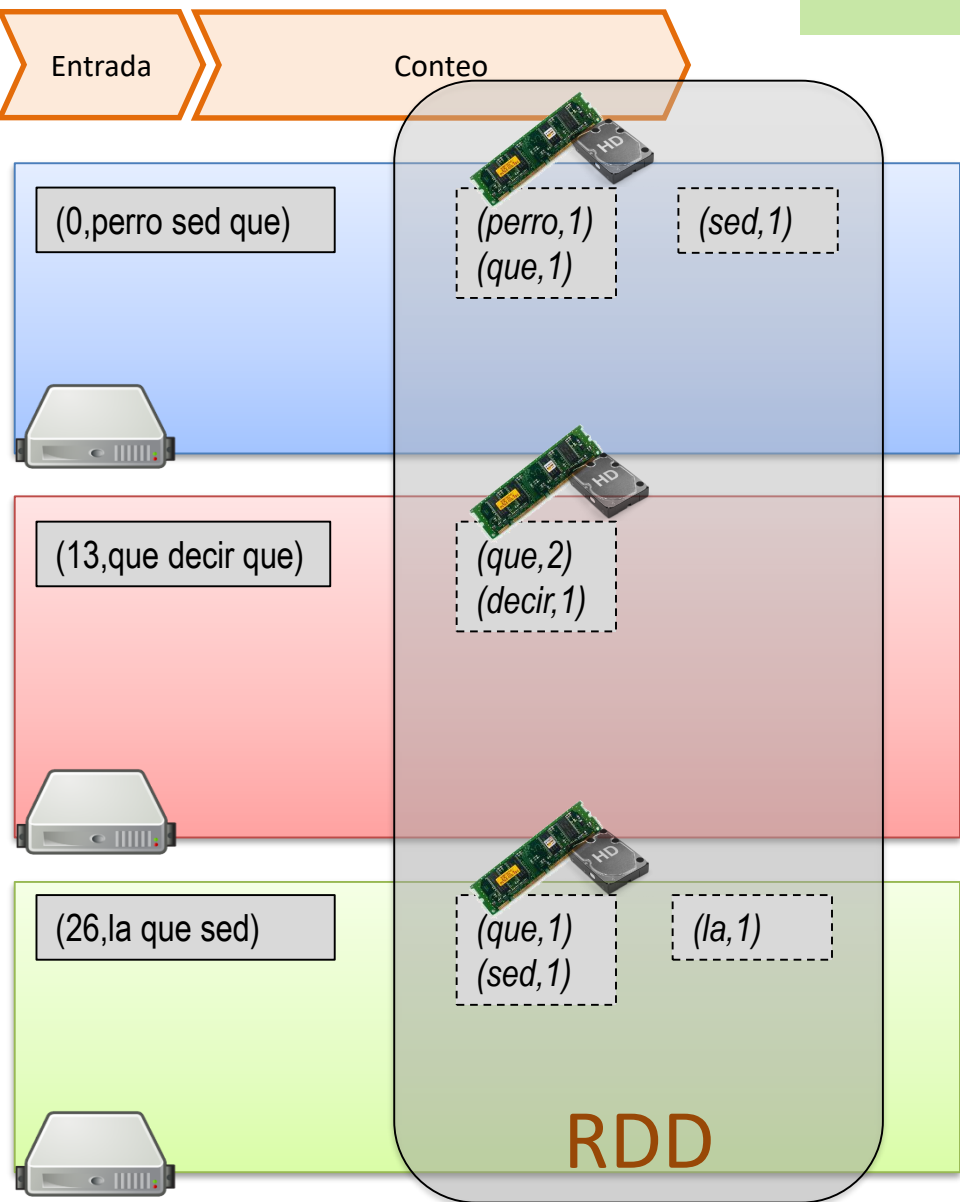
(perro,1)
(la,1)
(que,4)
(sed,2)
(decir,1)

APACHE
Spark™

¿Y si las palabras no caben en memoria? ?

Los RDDs pueden usar el disco duro

RDDs pueden tener varias particiones virtuales en una máquina



Tipos de RDDs en Spark

- HadoopRDD
- FilteredRDD
- MappedRDD
- PairRDD
- ShuffledRDD
- UnionRDD
- PythonRDD
- DoubleRDD
- JdbcRDD
- JsonRDD
- SchemaRDD
- VertexRDD
- EdgeRDD
- CassandraRDD
- GeoRDD
- EsSpark

Tipos específicos de RDD permiten operaciones específicas

`PairRDD` tiene especial importancia para MapReduce

APACHE SPARK: EJEMPLO

Spark: Productos por hora



cliente412	1L_Leche	2014-03-31T08:47:57Z	\$900
cliente412	Nescafe	2014-03-31T08:47:57Z	\$2.000
cliente412	Nescafe	2014-03-31T08:47:57Z	\$2.000
cliente413	400g_Zanahoria	2014-03-31T08:48:03Z	\$1.240
cliente413	El_Mercurio	2014-03-31T08:48:03Z	\$3.000
cliente413	Gillette_Mach3	2014-03-31T08:48:03Z	\$3.000
cliente413	Santo_Domingo	2014-03-31T08:48:03Z	\$2.450
cliente413	Nescafe	2014-03-31T08:48:03Z	\$2.000
cliente414	Rosas	2014-03-31T08:48:24Z	\$7.000
cliente414	400g_Zanahoria	2014-03-31T08:48:24Z	\$9.230
cliente414	Nescafe	2014-03-31T08:48:24Z	\$2.000
cliente415	1L_Leche	2014-03-31T08:48:35Z	\$900
cliente415	300g_Frutillas	2014-03-31T08:48:35Z	\$830

...

(cliente,producto,tiempo,valor)

¿Número de clientes comprando cada producto con valor > 1000 por hora del día?



Spark: Productos por hora



c1	p1	08	900
c1	p2	08	2000
c1	p2	08	2000
c2	p3	09	1240
c2	p4	09	3000
c2	p5	09	3000
c2	p6	09	2450
c2	p2	09	2000
c3	p7	08	7000
c3	p8	08	9230
c3	p2	08	2000
c4	p1	23	900
c4	p9	23	830

...

(cliente, producto, hora, valor)

¿Número de clientes comprando cada producto con valor > 1000 por hora del día?



load

filter($v > 1000$)

coalesce(3)

...

c1,p1,08,900
c1,p2,08,2000
c1,p2,08,2000

c4,p1,23,900
c4,p9,23,830

c1,p1,08,900
c1,p2,08,2000
c1,p2,08,2000

c4,p1,23,900
c4,p9,23,830

c1,p2,08,2000
c1,p2,08,2000

c1,p2,08,2000
c1,p2,08,2000

c2,p3,09,1240
c2,p4,09,3000
c2,p5,09,3000
c2,p6,09,2450
c2,p2,09,2000

c2,p3,09,1240
c2,p4,09,3000
c2,p5,09,3000
c2,p6,09,2450
c2,p2,09,2000

c2,p3,09,1240
c2,p4,09,3000
c2,p5,09,3000
c2,p6,09,2450
c2,p2,09,2000

c2,p3,09,1240
c2,p4,09,3000
c2,p5,09,3000
c2,p6,09,2450
c2,p2,09,2000

c3,p7,08,7000
c3,p8,08,9230
c3,p2,08,2000

c3,p7,08,7000
c3,p8,08,9230
c3,p2,08,2000

c3,p7,08,7000
c3,p8,08,9230
c3,p2,08,2000

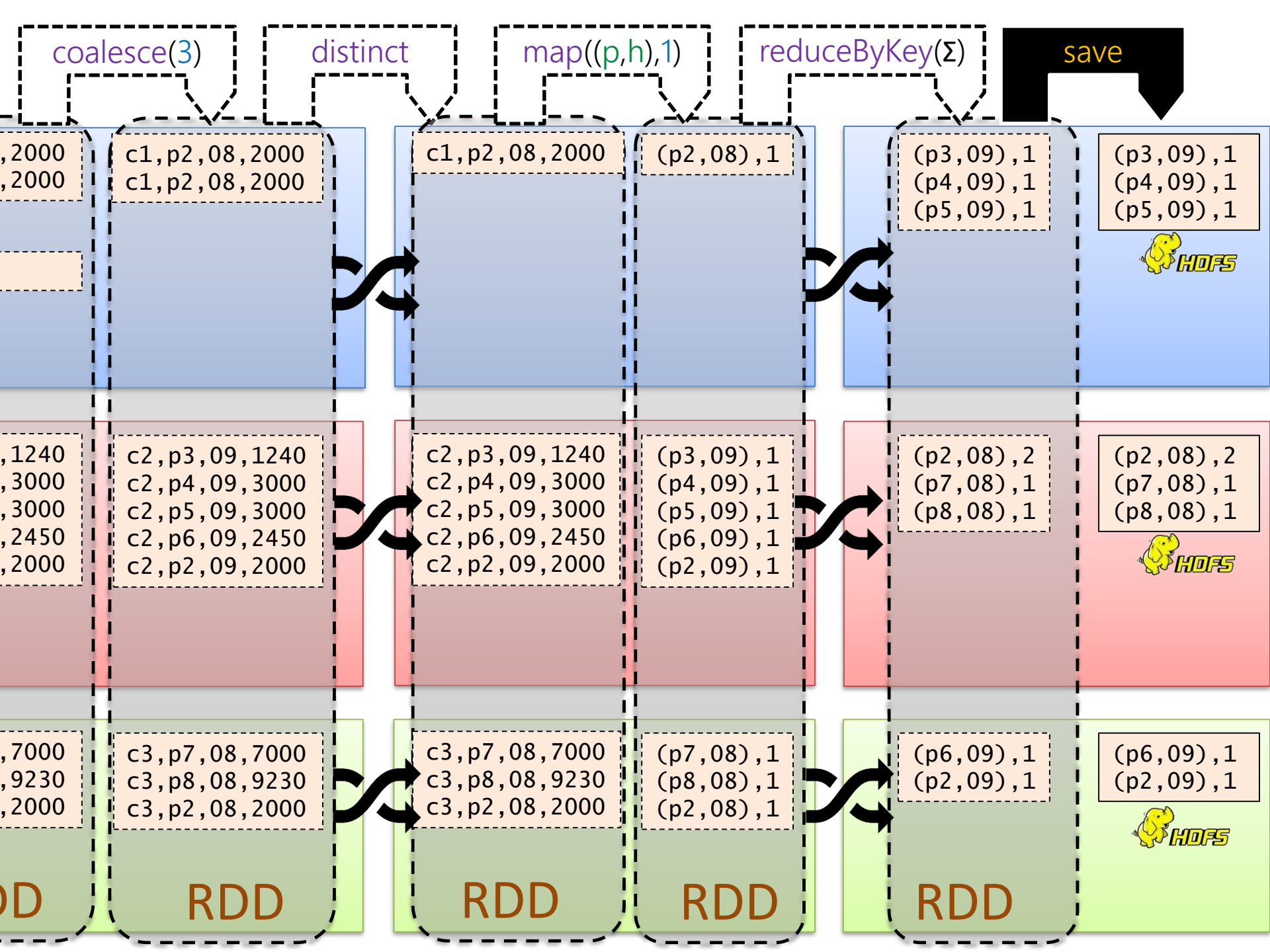
c3,p7,08,7000
c3,p8,08,9230
c3,p2,08,2000

RDD

RDD

RDD



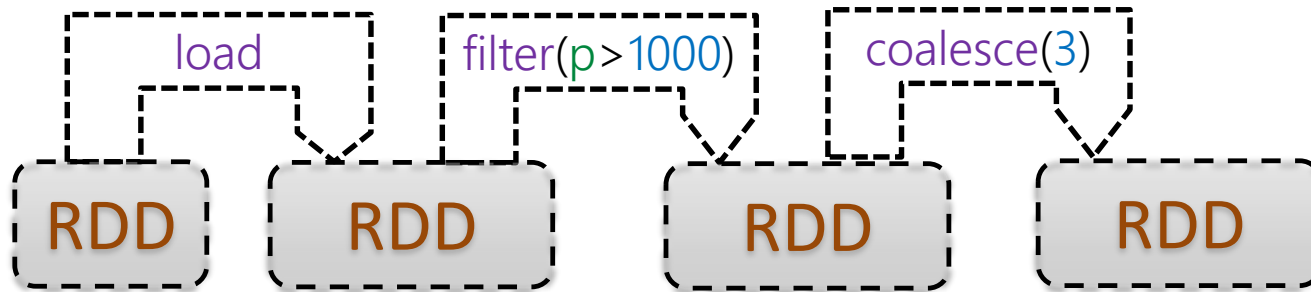


APACHE SPARK:

TRANSFORMACIONES Y ACCIONES

Spark: Transformaciones vs. Acciones

Las **Transformaciones** se ejecutan perezosamente ...
... resultan en RDDs virtuales
... se ejecutan sólo para completar una **acción**
... por ejemplo:



... no se ejecutan de inmediato

Transformaciones

La mayoría requieren un **PairRDD** ...

`R.map(f)`

`R.flatMap(f)`

`R.filter(f)`

`R.mapPartitions(f)`

`R.mapPartitionsWithIndex(f)`

`R.sample(.,.,.)`

`R.union(S)`

`R.intersection(S)`

`R.distinct()`

`R.groupByKey()`

`R.reduceByKey(f)`

`R.aggregateByKey(.,f,f')`

`R.sortByKey()`

`R.join(S)`

R.transform()

f = argumento de función

S = argumento de RDD

. = argumento simple

`R.cogroup(S)`

`R.cartesian(S)`

`R.pipe(.)`

`R.coalesce(.)`

`R.repartition(.)`

...

...

¿Por qué algunas están subrayadas?



Pueden requerir una transmisión de datos



Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

Transformación	Descripción	Ejemplo / Nota
<code>R.map(f)</code>	Mapear un elemento de R a un valor de salida	$f : (a, b, c) \mapsto (a, c)$
<code>R.flatMap(f)</code>	Mapear un elemento de R a cero o más valores de salida	$f : (a, b, c) \mapsto \{(a, b), (a, c)\}$
<code>R.filter(f)</code>	Mapear cada elemento de R que satisfaga f a la salida	$f : a > b$
<code>R.mapPartitions(f)</code>	Mapear todos los elementos de R a la salida en una llamada de f por partición	$f : R \rightarrow \pi_{a,c}(R)$
<code>R.mapPartitionsWithIndex(f)</code>	Como <code>mapPartitions</code> pero f tiene como argumento un índice indicando la partición	
<code>R.sample(w, f, s)</code>	Toma una muestra de R	w: con reemplazo f: fracción s: semilla

Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

R.union(*S*)

$R \cup S$

R.intersection(*S*)

$R \cap S$

R.distinct()

Borra duplicados

Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

R.groupByKey()

Agrupar valores por llave

R.reduceByKey(f)

Agrupar valores por llave y llama f para combinar y reducir valores de la misma llave

$f : (a, b) \mapsto a + b$

R.aggregateByKey(c, fc, fr)

Agrupar valores por llave usando c como un valor inicial, fc como un combiner y fr como una reducción

c : valor inicial

fc : combiner

fr : reducción

R.sortByKey([a])

Ordena por llave

a : true ascendente
false descendente

R.join(S)

$R \bowtie S$, join por llave

hay leftOuterJoin,
rightOuterJoin, y
fullOuterJoin

R.cogroup(S)

Agrupar valores por llave en R y S juntos

Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

R.cartesian(*S*)

R × *S*, producto cartesiano

R.pipe(*c*)

Hace "pipe" de **stdin** para procesar los datos con un comando *c* como **grep**, **awk**, **perl**, etc. El resultado es un RDD con las líneas de salida.

R.coalesce(*n*)

Hace un "merge" de varias particiones a *n* particiones

R.repartition(.)

Hace la partición de nuevo para balancear los datos

...

...

...

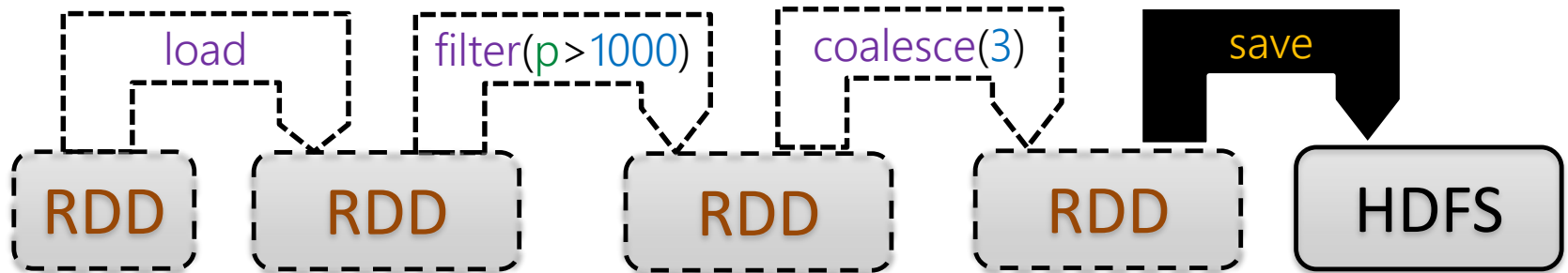
Spark: Transformaciones vs. Acciones

Las **Acciones** ejecutan los procesos ...

... resultan en RDDs materializados

... se ejecuta cada transformación subordinada

... por ejemplo:



... se ejecutan todos los pasos

... pero no se guardan los RDDs intermedios

Acciones

R.action()

f = argumento de función
.
= argumento simple

R.reduce(*f*)

R.collect()

R.count()

R.first()

R.take(.)

R.takeSample(.,.)

R.takeOrdered(.)

R.saveToCassandra()

R.saveAsTextFile(.)

R.saveAsSequenceFile(.)

R.saveAsObjectFile(.)

R.countByKey()

R.foreach(*f*)

...

Acciones

R.action()

f = argumento de función
.
= argumento simple

Acciones	Descripción	Ejemplo / Nota
<code>R.reduce(f)</code>	Reduce <u>todos</u> los datos a un valor (<u>no se hace por llave</u>)	$f : (a, b) \mapsto a + b$
<code>R.collect()</code>	Carga R como un arreglo en la aplicación local	
<code>R.count()</code>	Cuenta los elementos de R	
<code>R.first()</code>	Devuelve el primer valor de R	
<code>R.take(n)</code>	Devuelve un arreglo de los primeros n valores de R	
<code>R.takeSample(w, n, s)</code>	Devuelve un arreglo con una muestra de R	w : con reemplazo n : número s : semilla

Acciones

R.action()

f = argumento de función
.*.* = argumento simple

<code>R.saveAsTextFile(d)</code>	Guarda los datos en el sistema de archivos	d: directorio
<code>R.saveAsSequenceFile(d)</code>	Guarda los datos en el sistema de archivos con el formato optimizado SequenceFile de Hadoop	d: directorio
<code>R.saveAsObjectFile(d)</code>	Guarda los datos en el sistema de archivos usando los métodos de serialización de Java	
<code>R.countByKey()</code>	Cuenta los valores para cada llave	Solo con PairRDD
<code>R.foreach(<i>f</i>)</code>	Ejecuta la función <i>f</i> para cada elemento de R , típicamente para interactuar con algo externo	<i>f</i> : <code>println(r)</code>

APACHE SPARK:

TRANSFORMACIONES CON **PairRDD**

Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

R.groupByKey()

Agrupar valores por llave

R.reduceByKey(f)

Agrupar valores por llave y llama f para combinar y reducir valores de la misma llave

$f : (a, b) \mapsto a + b$

R.aggregateByKey(c, fc, fr)

Agrupar valores por llave usando c como un valor inicial, fc es un combiner y fr es una reducción

c : valor inicial

fc : combiner

fr : reducción

R.sortByKey([a])

Ordenar por llave

a : true ascendente
false descendente

¿Cómo son diferentes?

$R \bowtie S$, join por llave

hay leftOuterJoin,
rightOuterJoin, y
fullOuterJoin 

R.cogroup(S)

Agrupar valores por llave en R y S juntos

Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

`R.groupByKey()`

Agrupar valores por llave

`R.reduceByKey( $f$ )`

Agrupar valores por llave y llama f para combinar y reducir valores de la misma llave

$f : (a, b) \mapsto a + b$

`R.aggregateByKey( $c, fc, fr$ )`

Agrupar valores por llave usando c como un valor inicial, fc es un combiner y fr es una reducción

c : valor inicial
 fc : combiner
 fr : reducción

¿Para sumar los valores para cada llave de R ?



(1) `R.groupByKey().map((k, {v1, ..., vn})  $\mapsto$  (k, sum({v1, ..., vn})))`;

(2) `R.reduceByKey((u, v)  $\mapsto$  u + v)`;

(3) `R.aggregateByKey(0, (u, v)  $\mapsto$  u + v, (u, v)  $\mapsto$  u + v)`;

¡(2) tiene un combiner! (3) es igual pero menos conciso. ¡Entonces (2) es mejor!

Transformaciones

R.transform()

f = argumento de función

S = argumento de RDD

x = argumento simple

`R.groupByKey()`

Agrupar valores por llave

`R.reduceByKey( $f$ )`

Agrupar valores por llave y llama f para combinar y reducir valores de la misma llave

$f : (a, b) \mapsto a + b$

`R.aggregateByKey( $c, fc, fr$ )`

Agrupar valores por llave usando c como un valor inicial, fc es un combiner y fr es una reducción

c : valor inicial
 fc : combiner
 fr : reducción

¿Para promediar los valores para cada llave de R ?



(1) `R.groupByKey().map((k, {v1, ..., vn})  $\mapsto$  (k, avg({v1, ..., vn})))`;

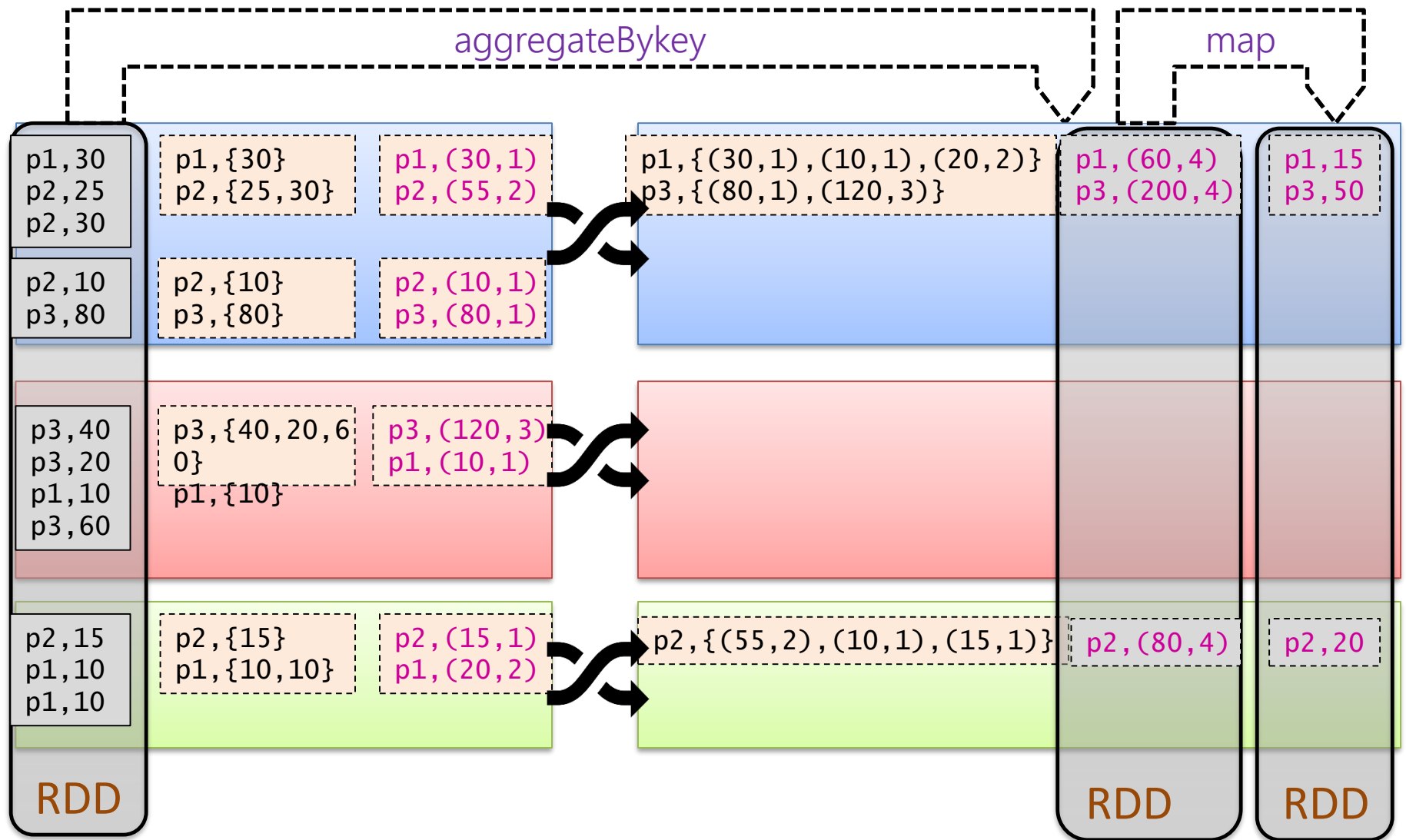
(2) `R1 = R.reduceByKey((u, v)  $\mapsto$  u + v)`;

`R2 = R.map((k, v)  $\mapsto$  (k, 1)).reduceByKey((u, v)  $\mapsto$  u + v)`;

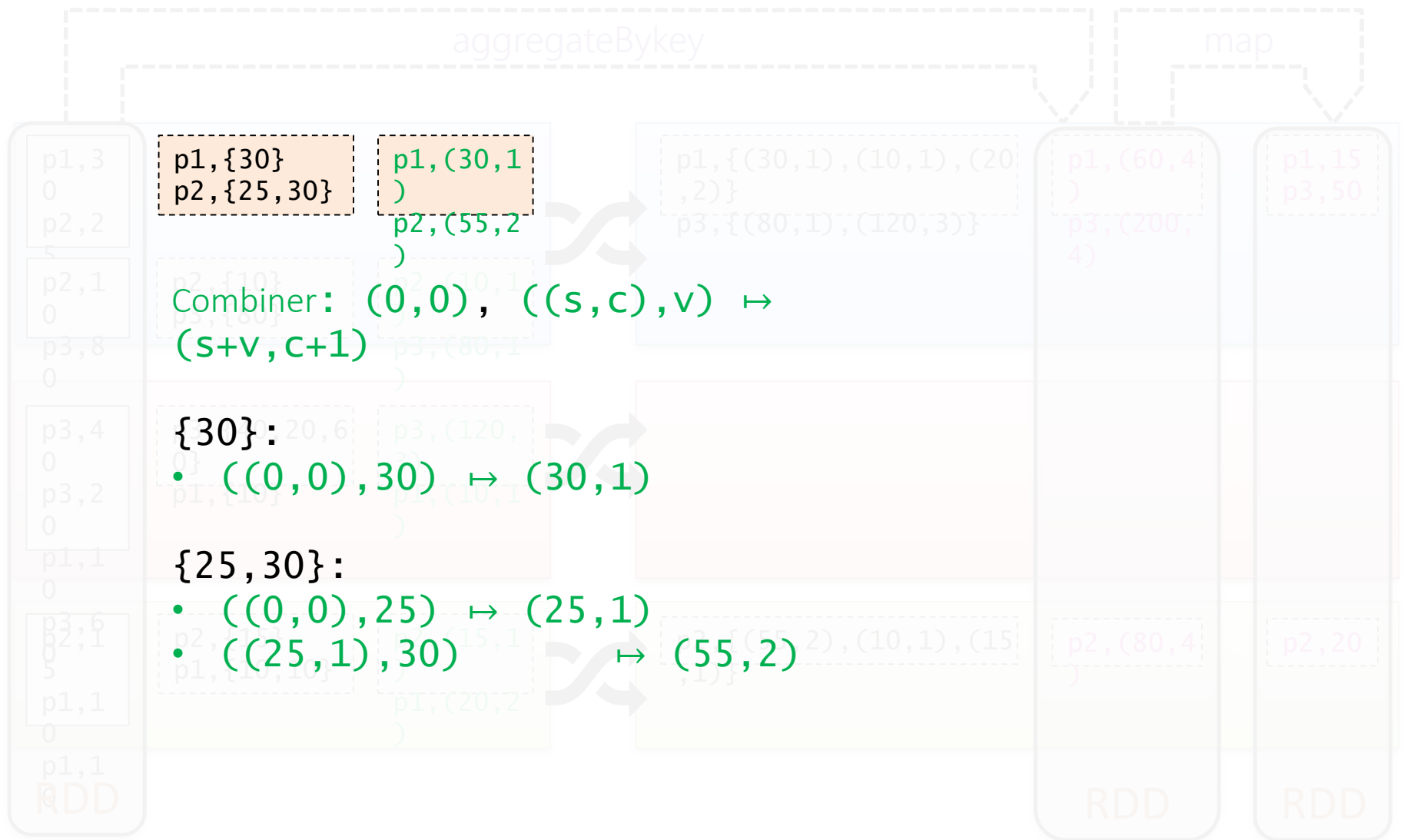
`R3 = R1.join(R2).map((k, (s, c))  $\mapsto$  (k, s/c))`;

(3) `R.aggregateByKey((0, 0), ((s, c), v) $\mapsto$ (s+v, c+1),
((s1, c1), (s2, c2)) $\mapsto$ (s1+s2, c1+c2))
.map((k, (s, c)) $\mapsto$ (k, s/c))`;

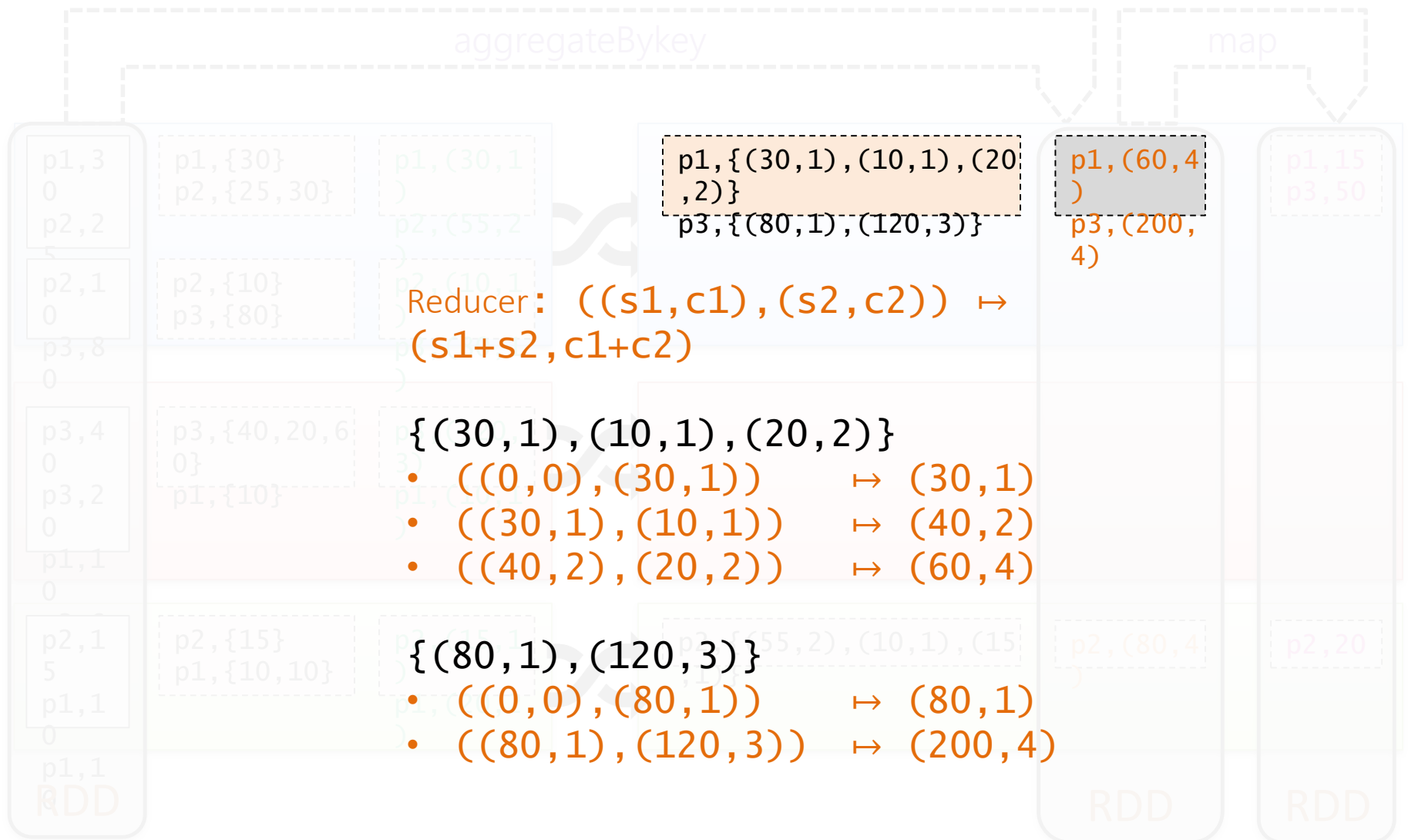
(3) tiene un combiner y necesita una sola transmisión. ¡Entonces (3) es mejor!



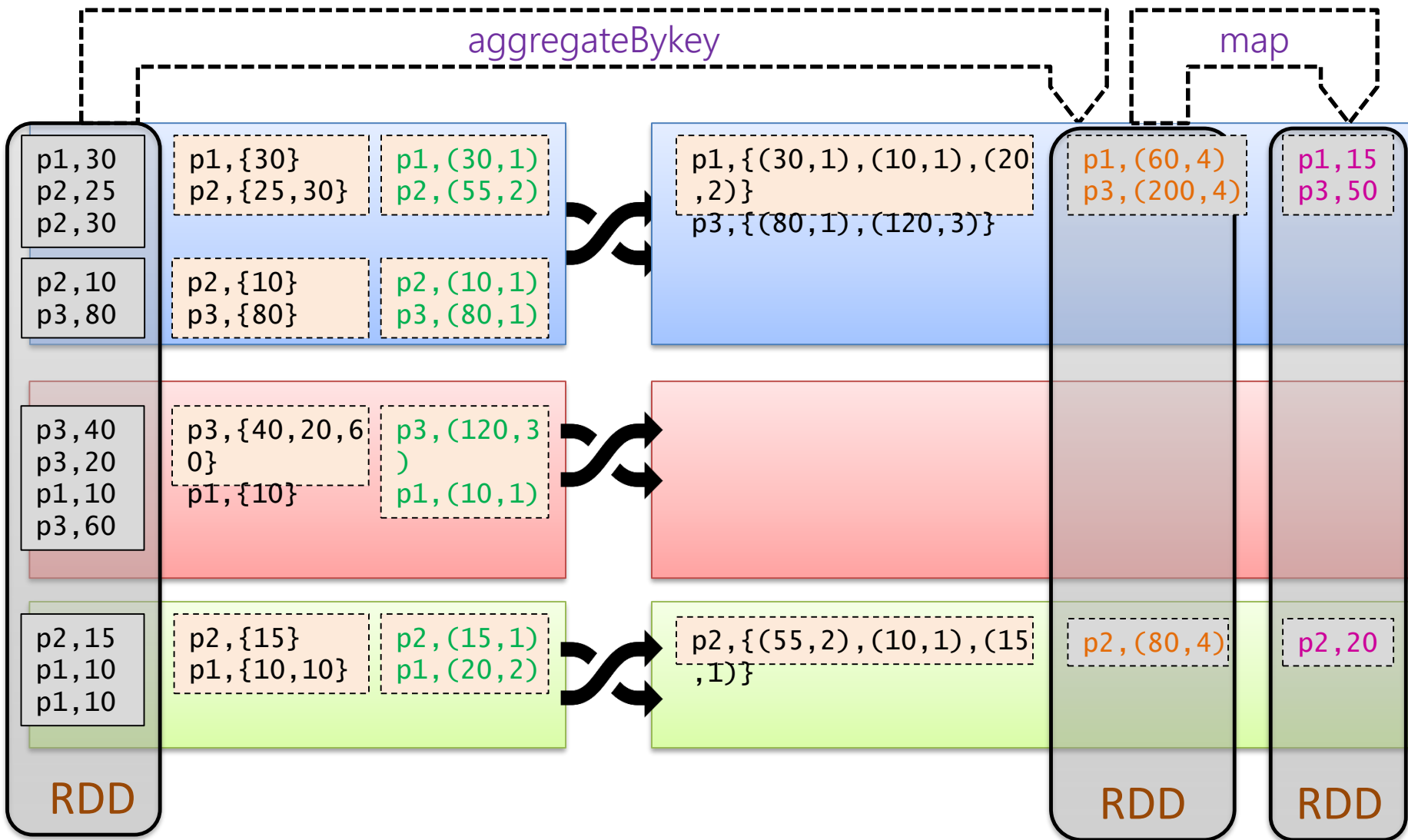
```
(3) R.aggregateByKey((0,0), ((s,c),v) ↦ (s+v,c+1),
                      ((s1,c1),(s2,c2)) ↦ (s1+s2,c1+c2))
    .map((k,(s,c)) ↦ (k,s/c));
```



```
(3) R.aggregateByKey((0, 0), ((s, c), v)  $\mapsto$  (s+v, c+1),
                      ((s1, c1), (s2, c2))  $\mapsto$  (s1+s2, c1+c2))
                      .map((k, (s, c))  $\mapsto$  (k, s/c));
```



```
(3) R.aggregateByKey((0,0), ((s,c),v) ↦ (s+v,c+1),
                      ((s1,c1),(s2,c2)) ↦ (s1+s2,c1+c2))
    .map((k,(s,c)) ↦ (k,s/c));
```

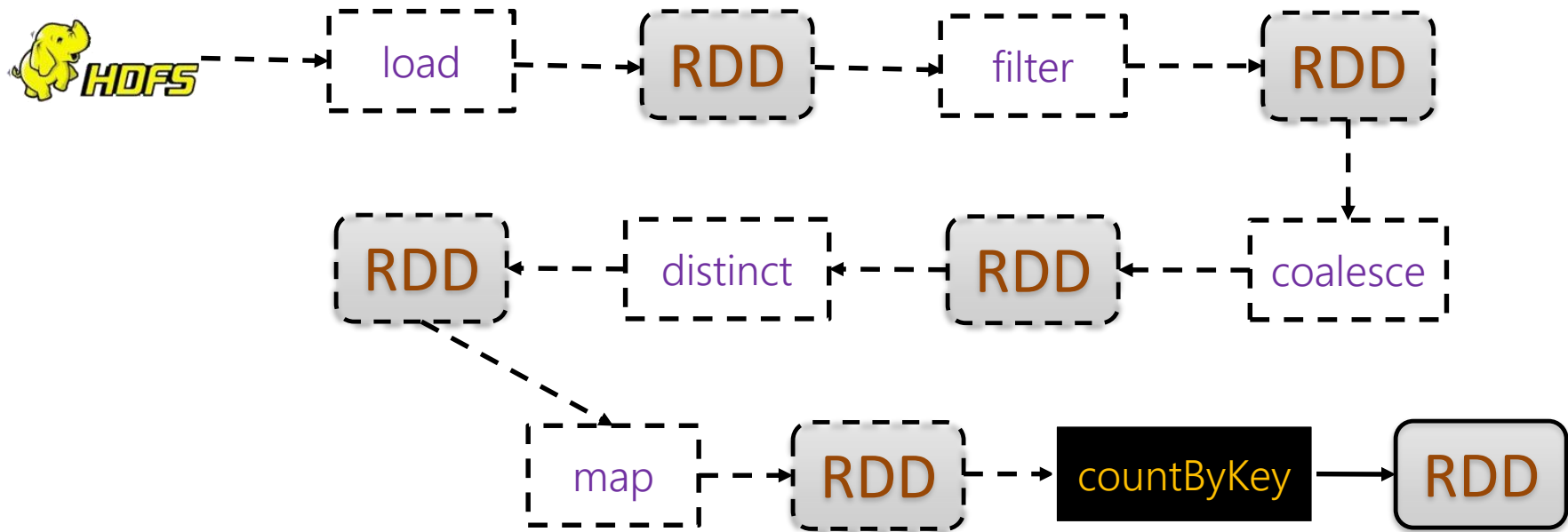


```
(3) R.aggregateByKey((0,0), ((s,c),v) ↦ (s+v,c+1),
                      ((s1,c1),(s2,c2)) ↦ (s1+s2,c1+c2))
      .map((k,(s,c)) ↦ (k,s/c));
```

APACHE SPARK:

"DIRECTED ACYCLIC GRAPH" ("DAG")

Spark: Grafo acíclico dirigido



Spark: Productos por hora

recibos.txt



c1	p1	08	900
c1	p2	08	2000
c1	p2	08	2000
c2	p3	09	1240
c2	p4	09	3000
c2	p5	09	3000
c2	p6	09	2450
c2	p2	09	2000
c3	p7	08	7000
c3	p8	08	9230
c3	p2	08	2000
c4	p1	23	900
c4	p9	23	830

... (cliente, producto, hora, valor)

clientes.txt



c1	fem	40
c2	mas	24
c3	fem	73
c4	fem	21

...

(cliente, género, edad)

¿Número de mujeres mayores de 30 comprando cada producto con valor > 1000 por hora del día?

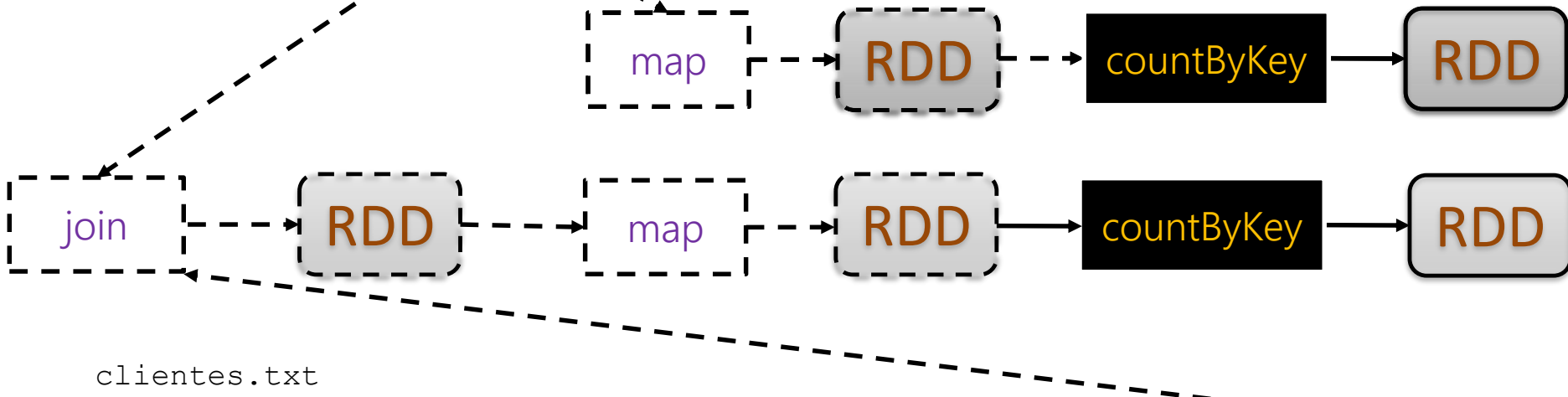
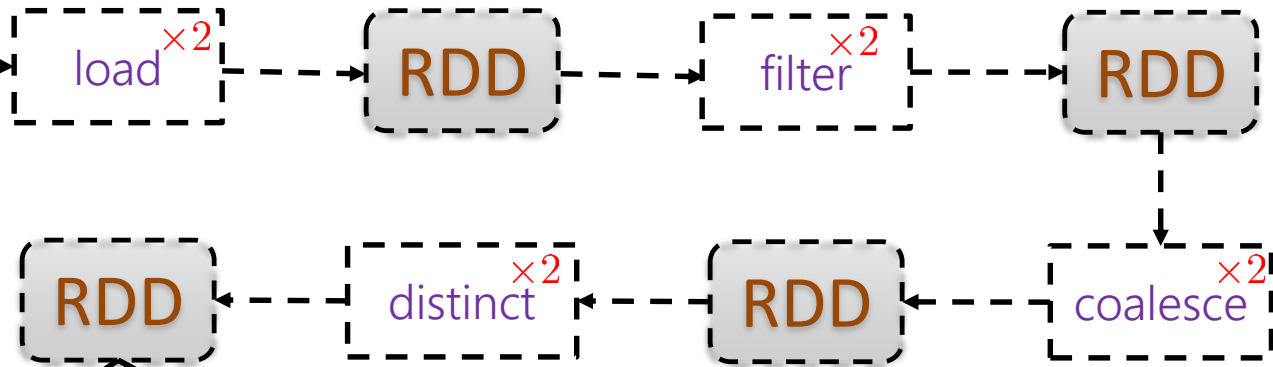


Spark: Grafo acíclico dirigido

Problema? 

Solución? 

recibos.txt



clientes.txt



Spark: Grafo acíclico dirigido

Problema? ?

Solución? ?

Materializar los **RDDs**
usados varias veces

recibos.txt



load

RDD

filter

RDD

cache

RDD

distinct

RDD

coalesce

RDD

map

RDD

countByKey

RDD

join

RDD

map

RDD

countByKey

RDD

clientes.txt

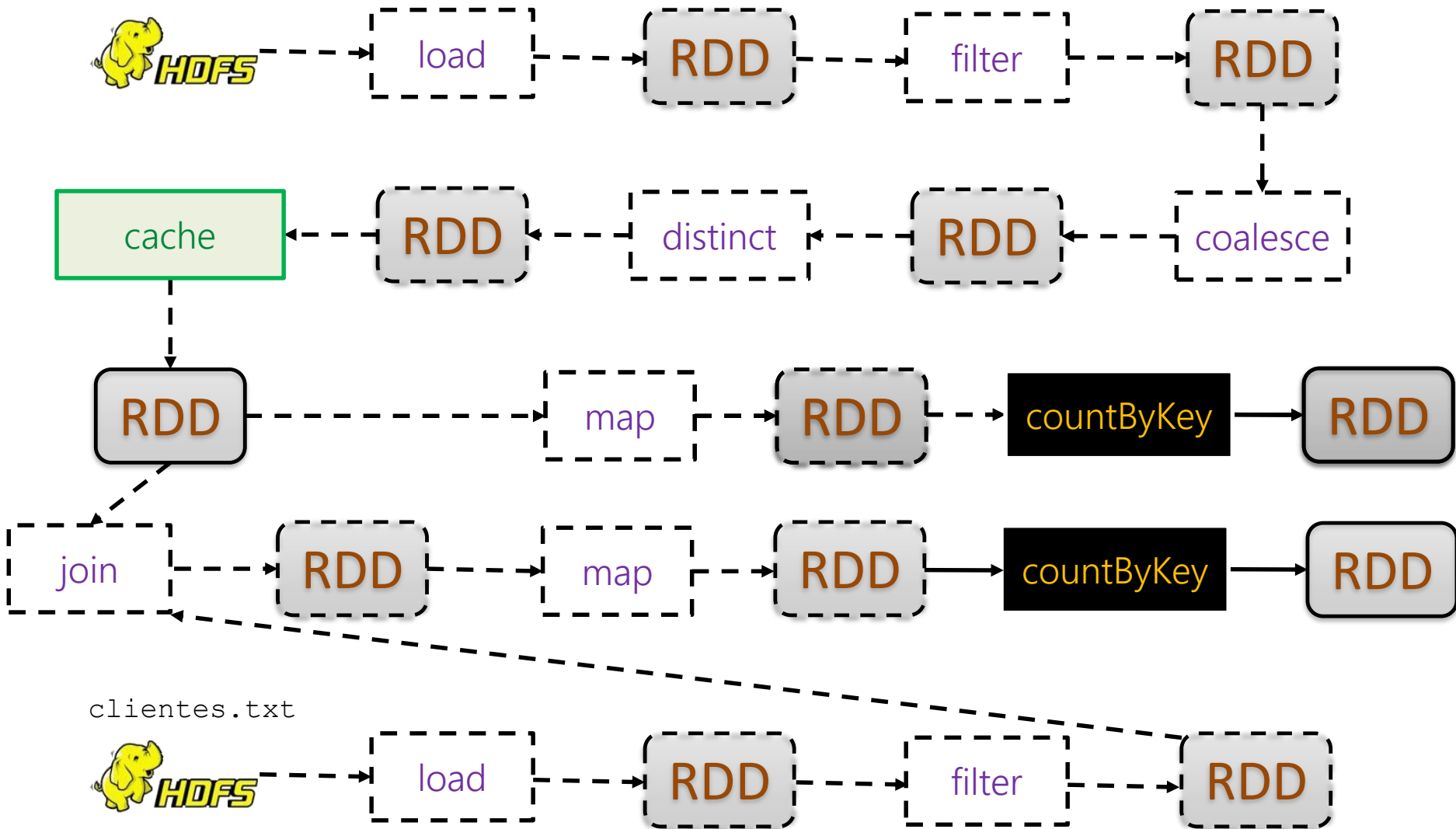


load

RDD

filter

RDD



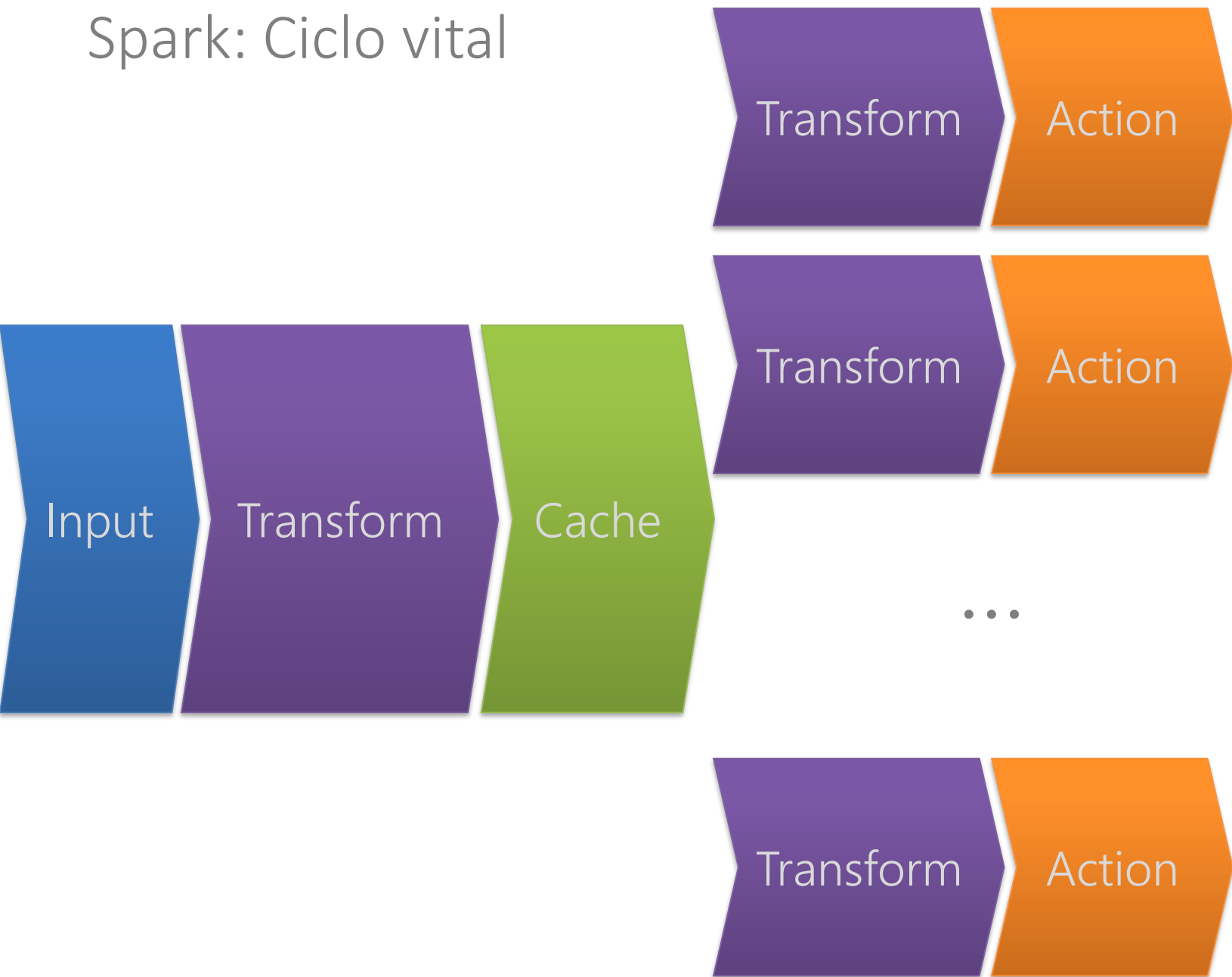
Spark: Grafo acíclico dirigido

- **cache** (también conocida como **persist**)
 - Perezoso (necesita una **acción** para ejecutarse)
 - Puede usar la memoria o el disco duro
 - (por defecto: usa sólo la memoria)

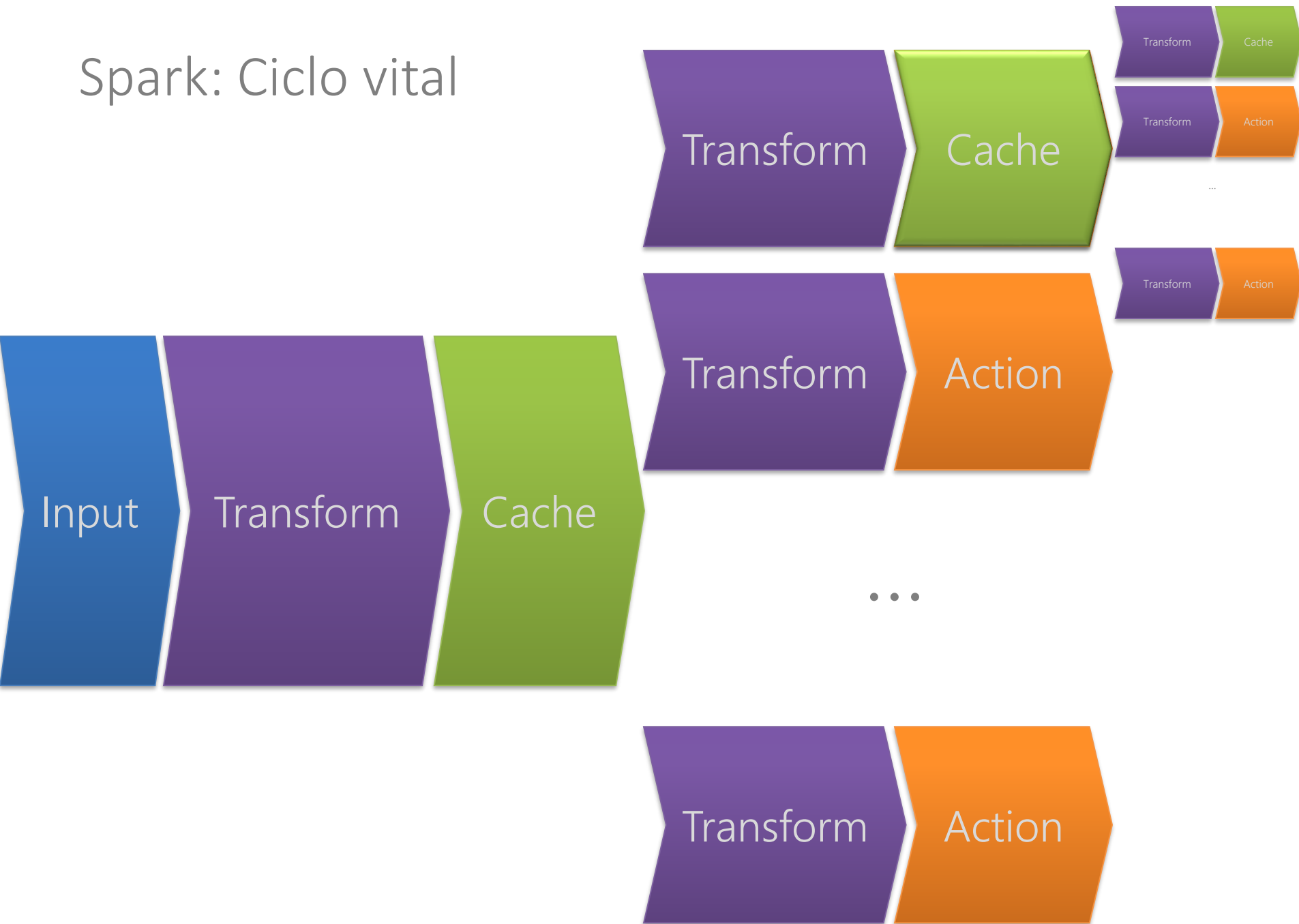
cache

APACHE SPARK: RESUMEN DEL CORE

Spark: Ciclo vital



Spark: Ciclo vital



Spark: Ciclo vital

- Leer **RDDs** como entrada
- **Transformar** **RDDs**
- **Cachear** **RDDs** usados varias veces
- Ejecutar una **acción** (y ejecutar las **transformaciones** subordinadas)
- Escribir la salida a un archivo / terminal / etc.

APACHE SPARK: MÁS ALLÁ DEL CORE

Hadoop vs. Spark: SQL, ML, Streams, ...



vs





SQL



MLlib



Streaming



Spark: Dataframes

- Permite abstraer los datos como tablas
- Algo parecido a Apache Pig/Hive
- Se pueden consultar los datos con Spark SQL

cust	item	time	price
customer412	Nescafe	2014-03-31T08:47:57Z	2000
customer412	Nescafe	2014-03-31T08:47:57Z	2000
customer413	400g_Zanahori a	2014-03-31T08:48:03Z	1240
customer413	Gillette_Mach 3	2014-03-31T08:48:03Z	8250
...	...	...	...



¿Preguntas?